UNIVERSIDAD PRIVADA “FRANZ TAMAYO”

FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS

“PRÁCTICA DE GIT – CONTROL DE VERSIONES”

Integrantes:

Michelle Jhovana Pari Mamani

Carlos Daniel Quea Chinche

Docente:

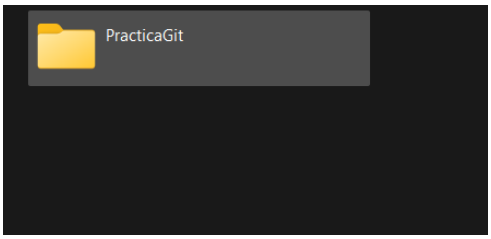
Ing. Patricia Fernandez Quisbert

EL ALTO - LA PAZ – BOLIVIA

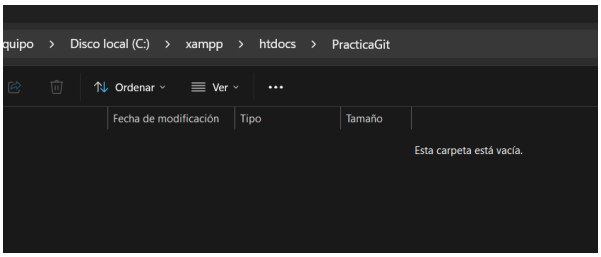
2026

A. Crear el repositorio

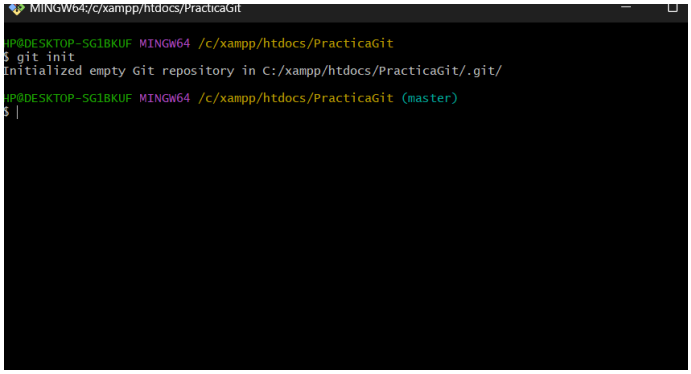
1. Cree una carpeta llamada: PracticaGit



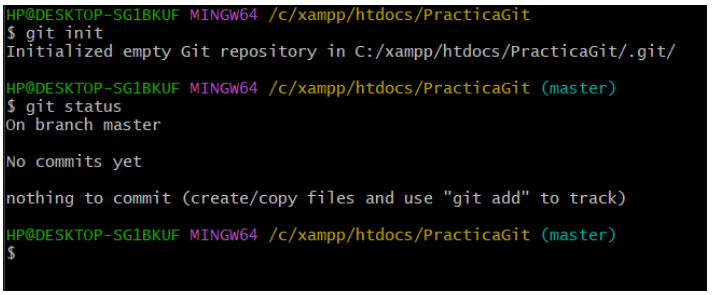
2. Ingrese a la carpeta.



3. Inicialice un repositorio Git.

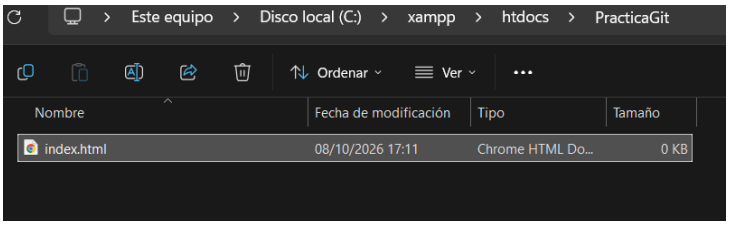


4. Verifique el estado del repositorio.



B. Área de trabajo y primer commit

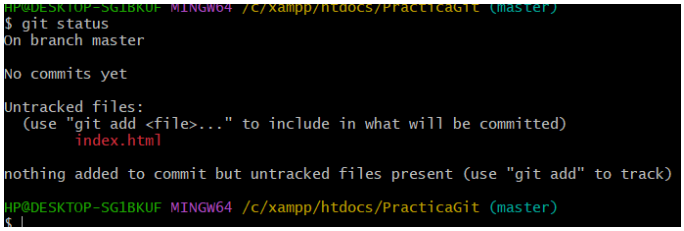
5. Cree un archivo llamado: index.html



Insertando el código en el archivo index.html



6. Consulte el estado del repositorio.



7 y 8. Agregue index.html al área de preparación (staging area). Y consultando nuevamente el estado.



9. Realice el primer commit con el mensaje: “Version inicial”

```
error: switch `m' requires a value

HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$ git commit -m" mi primera version "
[master (root-commit) 9135574]  mi primera version
 1 file changed, 12 insertions(+)
 create mode 100644 index.html

HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$
```

10. Consulte el historial.

```
HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$ git log
commit 913557457 36ecc92e1b35077edc54d9cb962572d (HEAD -> master)
Author: Michelle <eate.michellejhovana.pari.ma@unifranz.edu.bo>
Date:   Thu Oct 8 17:20:47 2026 -0400

    mi primera version

HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$
```

C. Modificar y guardar una nueva versión

11. Modifique index.html y agregue debajo del párrafo:

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Mi Perfil</title>
5  </head>
6  <body>
7
8      <h1>Ingeniería de Sistemas</h1>
9      <p>Estudiante: Su nombre</p>
10     <p>Materia: Proyecto integrador intermedio II</p>
11
12 </body>
13 </html>
```

12. Consulte el estado.

```
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")

HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$
```

13 y 14. Agregue el cambio al staging área y Realice un nuevo commit con el mensaje: “Version agrega materia”

```
P@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
 git add index.html

P@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
 git commit -m" version agrega materia"
master dd328e3]  version agrega materia
1 file changed, 1 insertion(+)

P@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
```

15. Consulte el historial.

```
HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$ git log
commit dd328e362d83270193445095087 82f0cbcc114ee (HEAD -> master)
Author: Michelle <eate.michellejhovana.pari.ma@unifranz.edu.bo>
Date:   Thu Oct 8 17:24:45 2026 -0400

     version agrega materia

commit 91355745736ecc92e1b35077edc54d9cb962572d
Author: Michelle <eate.michellejhovana.pari.ma@unifranz.edu.bo>
Date:   Thu Oct 8 17:20:47 2026 -0400

     mi primera version

HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$
```

D. Practicar restore

16. Modifique nuevamente index.html agregando

```
index.html M
C: > xampp > htdocs > PracticaGit > index.html > html > body
1   <!DOCTYPE html>
2   <html>
3   <head>
4       <title>Mi Perfil</title>
5   </head>
6   <body>
7
8       <h1>Ingeniería de Sistemas</h1>
9       <p>Estudiante: Su nombre</p>
10      <p>Materia: Proyecto integrador intermedio II</p>
11      <p>Este cambio será eliminado.</p>
12
13  </body>
14  </html>
```

17. Consulte el estado.

```
HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")

HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$
```

18. Use: git restore index.html

```
HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$ git restore index.html

HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$
```

19. Compruebe que el cambio desapareció.

index.html

C: > xampp > htdocs > PracticaGit > index.html > html

```
1   <!DOCTYPE html>
2   <html>
3   <head>
4       <title>Mi Perfil</title>
5   </head>
6   <body>
7
8       <h1>Ingeniería de Sistemas</h1>
9       <p>Estudiante: Su nombre</p>
10      <p>Materia: Proyecto integrador intermedio II</p>
11
12  </body>
13  </html>
```

Desaparecio la línea añadida

20. Consulte el estado del repositorio.

```
HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$ git status
On branch master
nothing to commit, working tree clean

HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$
```

Pregunta:

¿Cómo recupera este archivo? Origen, Destino

Seleccione entre: área de trabajo, área de preparación, repositorio

R.- El archivo se recupera desde el repositorio hacia el área de trabajo utilizando el comando git restore index.html. Esto elimina los cambios realizados que todavía no fueron guardados en un commit.

E. Crear y utilizar una rama

21. Consulte las ramas existentes: git branch

```
HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$ git branch
* master

HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$
```

22. Cree una rama llamada: diseño

```
HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$ git branch diseño

HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$ git branch
  diseño
* master

HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$
```

23. Cambie a esa rama. Puede hacerlo con: git checkout diseño

```
HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (master)
$ git checkout diseño
Switched to branch 'diseño'

HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (diseño)
$
```

24. Compruebe en qué rama se encuentra.

```
$ git branch
* diseño
  master

HP@DESKTOP-SG1BKUF MINGW64 /c/xampp/htdocs/PracticaGit (diseño)
$
```

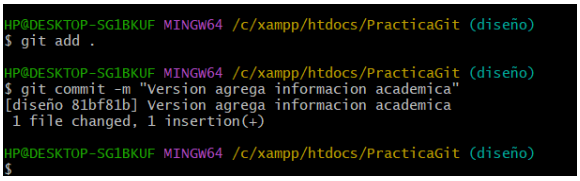
25. Modifique index.html agregando

index.html M

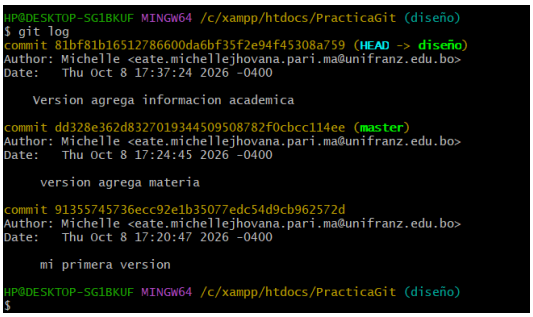
C: > xampp > htdocs > PracticaGit > index.html > html > body

```
1   <!DOCTYPE html>
2   <html>
3   <head>
4       <title>Mi Perfil</title>
5   </head>
6   <body>
7
8       <h1>Ingeniería de Sistemas</h1>
9       <p>Estudiante: Su nombre</p>
10      <p>Materia: Proyecto integrador intermedio II</p>
11      <h2>Mi información académica</h2>
12
13  </body>
14  </html>
```

26. Realice: git add . git commit -m "Version agrega informacion academica"



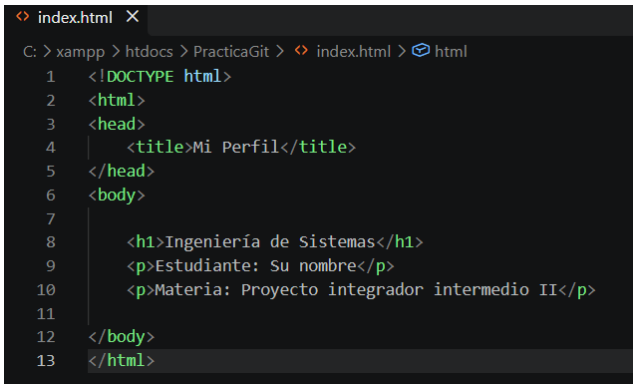
27. Consulte el historial.



28. Regrese a la rama principal: git checkout master Si Git creó la rama principal con el nombre main, utilice: git checkout main



29. Abra index.html. Observe: el cambio realizado en la rama diseño no aparece en la rama principal.

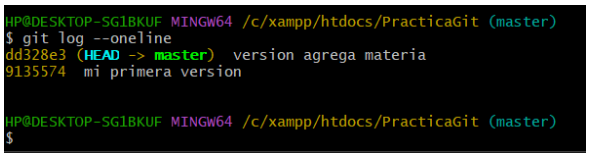


Pregunta: ¿Por qué la modificación realizada en diseño no aparece en master/main?

R.- Porque la modificación fue realizada y guardada en la rama diseño. Las ramas tienen historiales independientes, por lo que el cambio no aparece en master/main hasta que se integre mediante un merge.

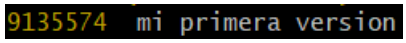
F. Recuperar información del historial

30. Ejecute: git log --oneline

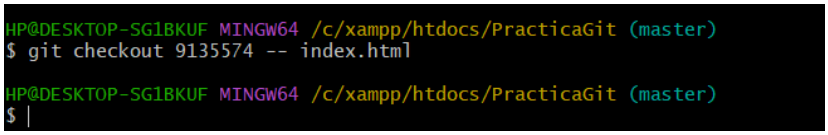


31. Identifique el código del commit correspondiente

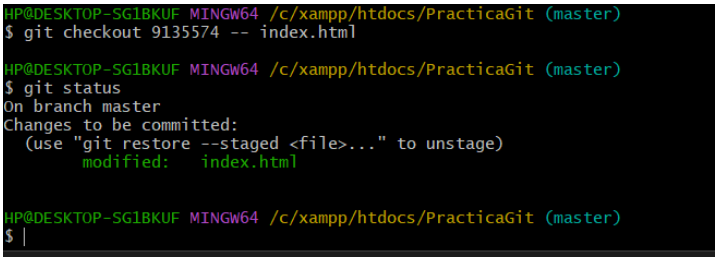
a:Version inicial



32. Use: git checkout CODIGO_COMMIT -- index.html Reemplace CODIGO_COMMIT por el código correspondiente.



33. Consulte el estado: git status



34. Observe qué ocurrió con index.html.



Pregunta:

¿Qué ocurrió con index.html? Explique

El archivo index.html volvió temporalmente al estado exacto que tenía en el commit de la "Version inicial" (perdiendo las líneas agregadas posteriormente). Al ejecutar git status, Git nos indica que el archivo ha sido modificado y se encuentra colocado directamente en el staging area (listo en color verde), esperando a ser confirmado o restaurado.

Preguntas finales. Responder de forma concreta:

1. ¿Cuál es la diferencia entre área de trabajo y staging area?

R.- El área de trabajo es donde se realizan y modifican los archivos. El staging area es donde se colocan los cambios con git add antes de realizar un commit.

2. ¿Para qué sirve git log --oneline?

R.- Sirve para mostrar de forma resumida el historial de commits, mostrando el código y el mensaje de cada commit.

3. ¿Qué diferencia existe entre git restore y git checkout?

R.- git restore sirve principalmente para descartar o recuperar cambios de archivos. git checkout sirve para cambiar de rama y también puede recuperar archivos desde un commit específico.

4. ¿Para qué sirve una rama (branch)?

R.- Una rama permite trabajar en cambios de forma independiente sin afectar directamente la rama principal. Luego los cambios pueden integrarse mediante un merge.